Applied Optimization
Assignment Module 1

First Name	
Last Name	
ID#	
Email Address	

Table of Contents

How to submit your Assignment 1

P1 – Rectangle with Greatest Area [10 points] 3

P2 - Cylindrical Water Tank using the Minimal Materials [15 points] 4

P3 - Type of the Optimization Problem [20 points] 6

P4 - Find the Global Maximum for Linear Optimization [20 points] 7

P5 - Find the Stationary Points and Inflection Points [20 points] 9

P6 – Convex Functions [20 points] 11

P7 – Python Coding [20 points] 13

Instructions for Students: How to submit your Assignment

After filling all the parts in this file, please follow the following steps.

1) Please download the file.
2) Add your name and ID to the first page.
3) Save the file in the original format (Docx or Doc)

   (please do not convert to other file formats e.g. PDF, ZIP, RAR, …).

4) Rename the file as

   *YOUR **First** Name - YOUR **Last** Name - ID - CSC252 - **HW1**.docx*

   Example: John - Smith - ID - CSC252 - HW1.docx

5) Upload the file and submit it (only using BrightSpace)

P1 – Rectangle with Greatest Area [10 points]

In about 300BC, Euclid proved that a square encloses the greatest area among all rectangles, assuming the total length of the four edges is fixed. Provide your own version of such proof.

Solution

AI USE DISCLAIMER & ACADEMIC INTEGRITY COMPLIANCE

In compliance with the **National University Student Guidelines for Generative AI**, this project discloses the following tool usage:

- **Conceptual Framing & Brainstorming (AI-Informed): Google Gemini** was utilized as a supplemental learning resource to clarify core mathematical and analytical concepts across the assignment questions.
- **Technical Execution & Boilerplate Generation (AI-Assisted):** The tool was used to generate initial code structures and programming boilerplates. These templates were subsequently modified, debugged, and tailored to meet the specific requirements of each question.
- **Human Oversight, Validation Loops & Refinement (Author-Owned):** The author maintained complete conceptual control, process oversight, and ownership of the entire homework scope. All AI-assisted templates, code logic, and conceptual outputs were systematically reviewed, rewritten, and validated by the author to ensure academic accuracy and integrity.
- **Disclosure Statement (AI-Generated): Google Gemini** was utilized to structure and audit this text format to fulfill all institutional reporting standards.

```
# define objective: `a square encloses the greatest area among all rectangles`
# define constraints: `assuming the total length of the four edges is fixed`

# constraints for rectangle: P(perimeter) = 2x + 2y
# y = P/2 - x

# define objective in terms of x.
# A(area) = x * y
# A = x * (P/2 - x)
# A = -x^2 + (P/2)x

# The result is a quadratic equation that can be solved where the maximum area is
# the highest point of the parabola (which is upside down since x^2 is negative.

# A = ax^2 + bx + c
# A = -1x^2 + (P/2)x + 0
# a = -1, b = (P/2)

# The maximum of a quadratic equation is:
# x = -b/2a
# x = -(P/2)/2(-1)
# x = P/4

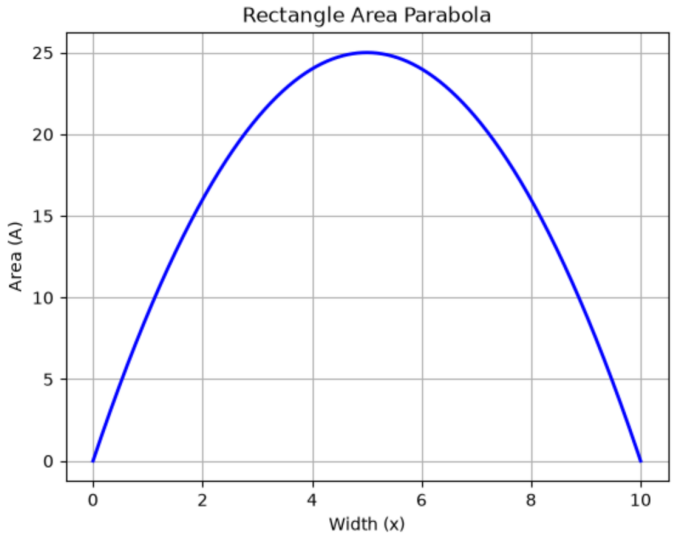
# Since the maximum of the curve is P/4, which is four equal sides, the proof shows that
# a square encloses the greatest area among all rectangles.

import matplotlib.pyplot as plt
import numpy as np
```

```
P = 20
x = np.linspace(0, P/2, 10000)
A = -x**2 + P/2 * x

plt.plot(x, A, color="blue", linewidth=2)
plt.title("Rectangle Area Parabola")
plt.xlabel("Width (x)")
plt.ylabel("Area (A)")
plt.grid(True)
plt.show()

max = np.argmax(A)
print(f'Max x = {x[max]:.2f}')
print(f'Max y = {(P / 2) - x[max]:.2f}')
print(f'Max A = {A[max]:.2f}')
```




```
Max x = 5.00
Max y = 5.00
Max A = 25.00
```

P2 - Cylindrical Water Tank using the Minimal Materials [15 points]

Design a cylindrical water tank which uses the minimal materials and holds the largest volume of water. What is the relationship between the radius r of the base and the height h?

Solution

```
# define objective: `a cylinder encloses the greatest volume with minimal material (surface area)`
# define constraints: size of radius (r) and height (h)

# Volume (V) of a cylinder: V = pi * r^2 * h
# Surface area (A) of a cylinder: A = 2 * pi * r^2 + 2 * pi * r * h
```

```
# Volume in terms of height (h): h = V / (pi * r^2)
# Surface area in terms of radius (r): A = 2 * pi * r^2 + 2 * pi * r * (V / pi * r^2)
# A = 2 * pi * r^2 + 2V / r

# Take the derivative to get the slope of the curve:
# dA/dr = 4 * pi * r - 2V / r^2

# Set the derivative to zero to get the minimum material/surface area
# 4 * pi * r - 2V / r^2 = 0
# 4 * pi * r = 2V / r^2
# 2 * pi * r^3 = V

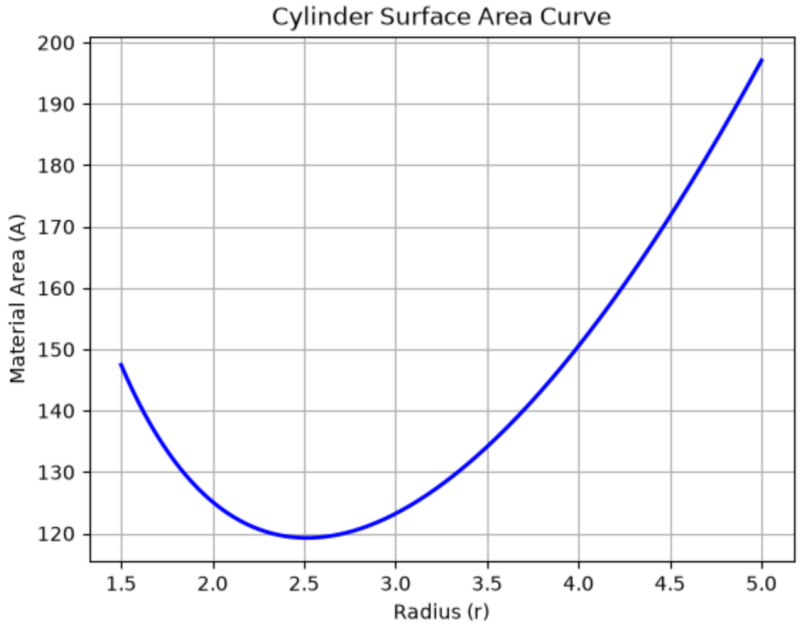
# Substitue volume equation for V in the optimal volume expression from above
# 2 * pi * r^3 = pi * r^2 * h
# 2r = h

# 2r is just the diameter so the relationship of optimal volume for r and h
# is when the diameter (2r) is the same as the height (h).

V = 100
r = np.linspace(1.5, 5, 10000)
A = 2 * np.pi * r**2 + (2 * V) / r

plt.plot(r, A, color="blue", linewidth=2)
plt.title("Cylinder Surface Area Curve")
plt.xlabel("Radius (r)")
plt.ylabel("Material Area (A)")
plt.grid(True)
plt.show()

min_idx = np.argmin(A)
print(f'Min r = {r[min_idx]:.2f}')
print(f'Min h = {V / (np.pi * r[min_idx]**2):.2f}')
print(f'Min A = {A[min_idx]:.2f}')
```




```
Min r = 2.52
Min h = 5.03
Min A = 119.27
```

P3 - Type of the Optimization Problem [20 points]

State the type of the following optimization problems:

a) max *exp(-x2)* subject to *-2 < x < 5*;

b) min $x^2 + y^2$;

c) max 2x + 3y subject to *x > 0, 0 < y < 10, y > x*;

d) min x^2y^6 subject to $x^2 + y^2 / 4 = 1$;

e) c

f) Design a software package to predict the stock market.

Solution

a) max exp(-x^2) subject to -2 < x < 5
Single-objective (one function), constrained (bounded -2 < x < 5), unimodal (single peak for x), non-linear (-x^2), continuous (x can be non-discrete values), and deterministic.

b) min x^2 + y^2
Single-objective (one function), unconstrained, unimodal (u-shaped parabola/quadratic), non-linear (quadratic), continuous, deterministic

c) max 2x + 3y subject to x > 0, 0 < y < 10, y > x
Single-objective, constrained (three bounds x > 0, 0 < y < 10, y > x), unimodal, linear (y = mx + b), continuous, deterministic

d) min x^2 * y^6 subject to x^2 + y^2 / 4 = 1

Single-objective, constrained (by equation x^2 + y^2 / 4 = 1), multi-modal (polynomial equation with equation constraint will have multiple local min/max and a global min/max), non-linear (polynomial), continuous, deterministic

e) Design a quieter, more efficient, low cost car engine
Multi-objective (objective for noise, efficiency, and cost), constrained (limits of parts, cost, noise, labor, etc.), multimodal (multiple variables that will create many local min/max and likely an unknown global min/max), non-linear, discrete and continuous (cylinder count (e.g. inline-4, v6, v8) versus cylinder size (e.g. 3.0, 4.1)), deterministic

f) Design a software package to predict the stock market
Multi-objective (stock price, margin, market cap, etc.), constrained (cost to predict, funds available, risk aversion, etc.), multimodal (infinite variables possible with infinite local min/max and definitely an unknown global min/max), non-linear, continuous and discrete variables (although mostly continuous as even stock can be sold in partials), stochastic.

- a) max exp(-x^2) subject to -2 < x < 5
  - Single-objective (one function), constrained (bounded -2 < x < 5), unimodal (single peak for x), non-linear (-x^2), continuous (x can be non-discrete values), and deterministic.
- b) min x^2 + y^2
  - Single-objective (one function), unconstrained, unimodal (u-shaped parabola/quadratic), non-linear (quadratic), continuous, deterministic
- c) max 2x + 3y subject to x > 0, 0 < y < 10, y > x
  - Single-objective, constrained (three bounds x > 0, 0 < y < 10, y > x), unimodal, linear (y = mx + b), continuous, deterministic
- d) min x^2 * y^6 subject to x^2 + y^2 / 4 = 1
  - Single-objective, constrained (by equation x^2 + y^2 / 4 = 1), multi-modal (polynomial equation with equation constraint will have multiple local min/max and a global min/max), non-linear (polynomial), continuous, deterministic
- e) Design a quieter, more efficient, low cost car engine
  - Multi-objective (objective for noise, efficiency, and cost), constrained (limits of parts, cost, noise, labor, etc.), multimodal (multiple variables that will create many local min/max and likely an unknown global min/max), non-linear, discrete and continuous (cylinder count (e.g. inline-4, v6, v8) versus cylinder size (e.g. 3.0, 4.1)), deterministic
- f) Design a software package to predict the stock market
  - Multi-objective (stock price, margin, market cap, etc.), constrained (cost to predict, funds available, risk aversion, etc.), multimodal (infinite variables possible with infinite local min/max and definitely an unknown global min/max), non-linear, continuous and discrete variables (although mostly continuous as even stock can be sold in partials), stochastic.

P4 - Find the Global Maximum for Linear Optimization [20 points]

Find the global maximum for linear optimization $f(x, y) = x+y$ subject to $0 < x < 5$, $0 < y < 2$ and $2x + y = 8$.

Solution

```
# ANA670 - Week 1 HW - Q4
# Find the global maximum for linear optimization
# f(x, y) = x+y
# subject to 0 < x < 5, 0 < y < 2 and 2x + y = 8

# Get all equations in terms of x:
# 2x + y = 8
# y = -2x + 8

# account for y constraints:
# 0 < -2x + 8 < 2
# 3 < x < 4 (new full x,y constraints
# because 3 < x < 4 falls within 0 < x < 5 and is more restrictive)

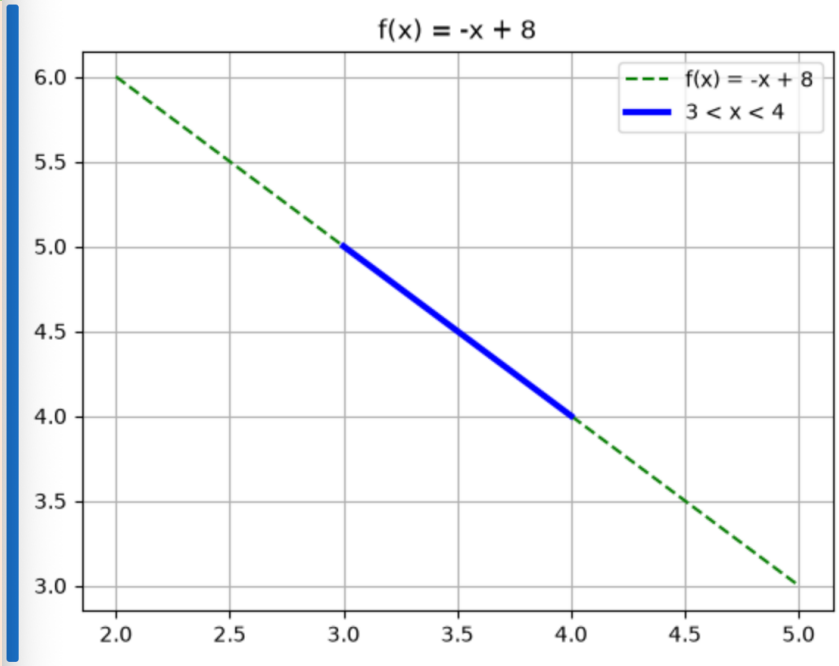
# substitute constraints back into main equation
# f(x) = x + y
# f(x) = x + -2x + 8
# f(x) = -x + 8

# This is a linear equation with a (-1) slope. That means the max value
# is the farthest we can go left within the constraints. At x = 3 we get a result of 5.
# Since the constraint is x > 3, we can never actually get to 3. We can have 3.01,
# 3.001 and keep going and adding infinite zeros without actually getting to 3.
# Because of this, the max value does not exist and there is no global max within the constraints.

x = np.linspace(2, 5, 100)
x_allowed = np.linspace(3, 4, 100)

plt.plot(x, -x + 8, "g--", label="f(x) = -x + 8")
plt.plot(x_allowed, -x_allowed + 8, "b-", linewidth=3, label="3 < x < 4")

plt.title("f(x) = -x + 8")
plt.grid(True)
plt.legend()
plt.show()
```



P5 - Find the Stationary Points and Inflection Points [20 points]

Find the stationary points and inflection points of the following functions.

a) $f(x) = \cos(x)$
b) $f(x) = exp(-x^2)$

Solution

```
# stationary points => first derivative where slope = 0
# inflection points => second derivative where slope changes from concave/convex

# a) f(x) = cos(x)
# stationary points:
# f'(x) = -sin(x) = 0
# x = n * pi for any integer n

# inflection points:
# f''(x) = -cos(x) = 0
# x = pi / 2 + n * pi for any integer n

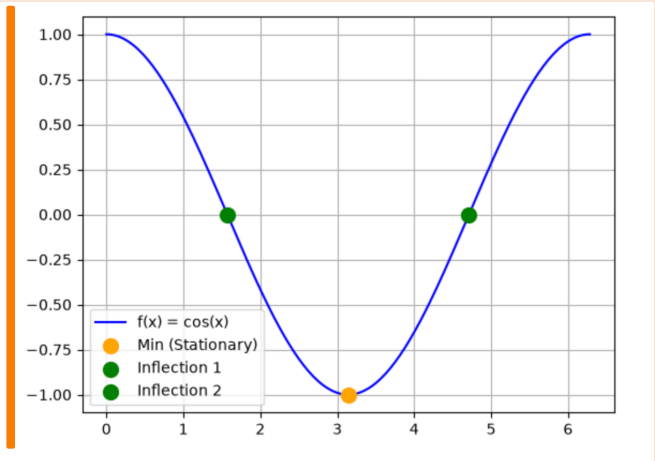
x = np.linspace(0, 2 * np.pi, 200)
y = np.cos(x)

points_x = [np.pi, np.pi/2, 3*np.pi/2]
points_y = [-1, 0, 0]
labels = ['Min (Stationary)', 'Inflection 1', 'Inflection 2']
colors = ['orange', 'green', 'green']

plt.plot(x, y, label='f(x) = cos(x)', color='blue')
for px, py, lbl, col in zip(points_x, points_y, labels, colors):
```

```
    plt.scatter(px, py, color=col, s=100, label=lbl, zorder=5)

plt.grid(True)
plt.legend()
plt.show()
```




```
# stationary points => first derivative where slope = 0
# inflection points => second derivative where slope changes from concave/convex

# b) f(x) = exp(-x^2)
# stationary points:
# f'(x) = -2x * exp(-x^2) = 0
# x = 0 (calculated using Gemini)

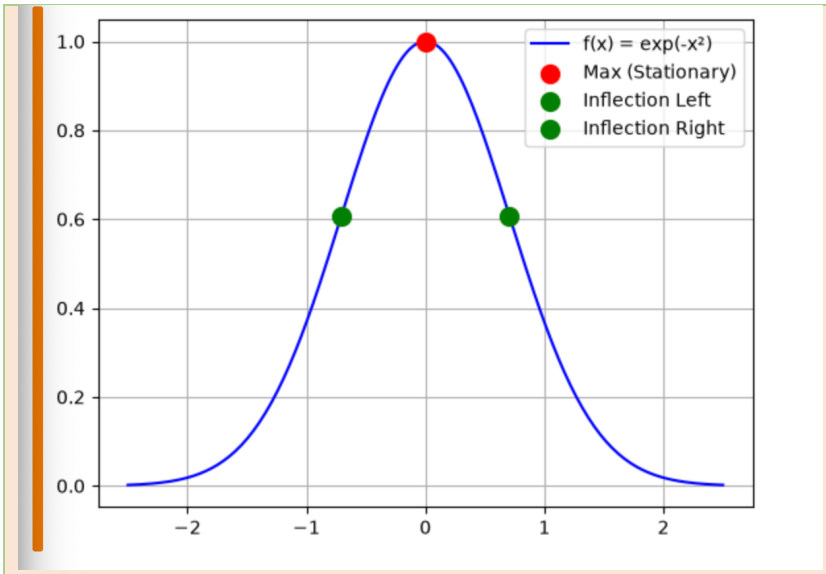
# inflection points:
# f''(x) = (4x^2 - 2) * exp(-x^2) = 0
# x = -1/sqrt(2) and x = 1/sqrt(2) (calculated using Gemini)

x = np.linspace(-2.5, 2.5, 200)
y = np.exp(-x**2)

points_x = [0, -1/np.sqrt(2), 1/np.sqrt(2)]
points_y = [1, np.exp(-0.5), np.exp(-0.5)]
labels = ['Max (Stationary)', 'Inflection Left', 'Inflection Right']
colors = ['red', 'green', 'green']

plt.plot(x, y, label='f(x) = exp(-x²)', color='blue')
for px, py, lbl, col in zip(points_x, points_y, labels, colors):
    plt.scatter(px, py, color=col, s=100, label=lbl, zorder=5)

plt.grid(True)
plt.legend()
plt.show()
```



P6 – Convex Functions [20 points]

Find if the following functions are convex or concave or neither?

a) $x^2+\exp(y)$;
b) $1-x^2-y^2$;
c) xy;
d) x^2/y for $y > 0$.

Solution

```
from matplotlib import cm

def compute_hessian(f, eq_string, x=0.0, y=1.0, h=1e-5, graph=True):
    df_dx = lambda x, y: (f(x + h, y) - f(x - h, y)) / (2 * h)
    df_dy = lambda x, y: (f(x, y + h) - f(x, y - h)) / (2 * h)

    f_xx = (df_dx(x + h, y) - df_dx(x - h, y)) / (2 * h)
    f_yy = (df_dy(x, y + h) - df_dy(x, y - h)) / (2 * h)
    f_xy = (df_dx(x, y + h) - df_dx(x, y - h)) / (2 * h)
    f_yx = (df_dy(x + h, y) - df_dy(x - h, y)) / (2 * h)

    matrix = np.round(np.array([[f_xx, f_xy], [f_yx, f_yy]]), decimals=4)
    if graph:
        fig = plt.figure(figsize=(6, 4))
        ax = fig.add_subplot(111, projection='3d')

        _x = np.linspace(-2, 2, 40)
        _y = np.linspace(0.1, 2, 40)
        X, Y = np.meshgrid(_x, _y)
        Z = np.vectorize(f)(X, Y)

        if "y" in eq_string and "/" in eq_string:
            Z = np.clip(Z, 0, 8)
```

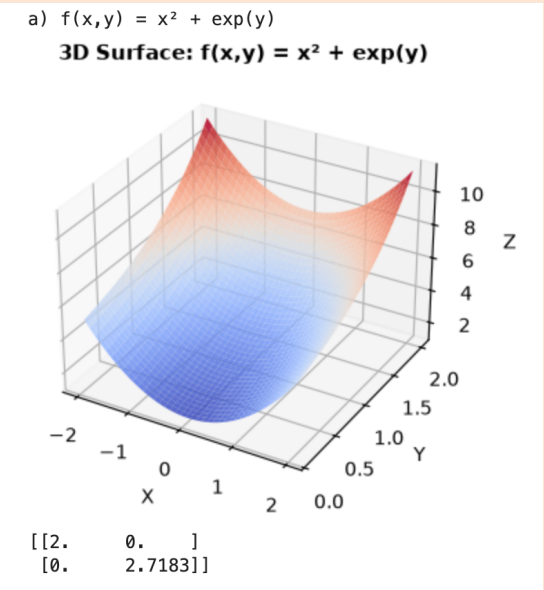
```
        ax.plot_surface(X, Y, Z, cmap=cm.coolwarm, edgecolor='none', alpha=0.85)
        ax.set_title(f"3D Surface: f(x,y) = {eq_string}", fontsize=11, fontweight='bold')
        ax.set_xlabel('X')
        ax.set_ylabel('Y')
        ax.set_zlabel('Z')
        plt.show()
    return matrix

equations = [
    ("a", lambda x, y: x**2 + np.exp(y), "x² + exp(y)"),
    ("b", lambda x, y: 1 - x**2 - y**2, "1 - x² - y²"),
    ("c", lambda x, y: x * y,           "xy"),
    ("d", lambda x, y: (x**2) / y if y != 0 else 0, "x² / y"),
]

for label, eq, eq_string in equations:
    print(f"{label}) f(x,y) = {eq_string}")
    print(compute_hessian(eq, eq_string, x=0.0, y=1.0, graph=True))
    print()

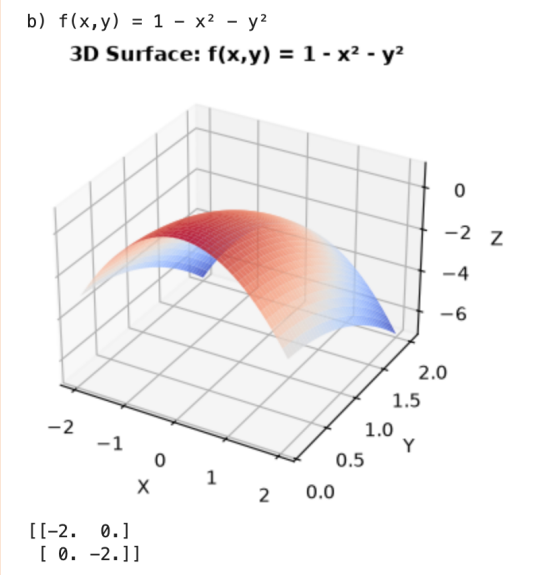
print(f'a) is Convex because all diagonal curvatures are positive.')
print(f'b) is Concave because all diagonal curvatures are negative.')
print(f'c) is Neither because there is a saddle point.')
print(f'd) is Convex because the matrix is positive where y > 0.')
```

```
a) f(x,y) = x² + exp(y)
```



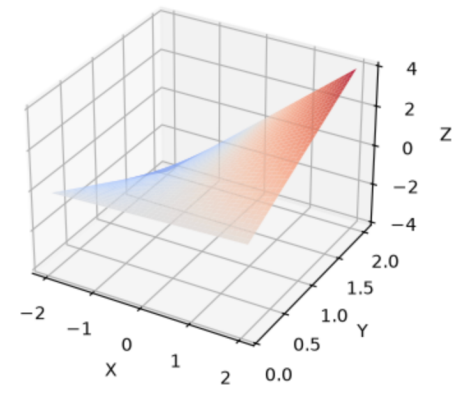

```
[[2.     0.    ]
 [0.     2.7183]]
```

```
b) f(x,y) = 1 - x² - y²
```



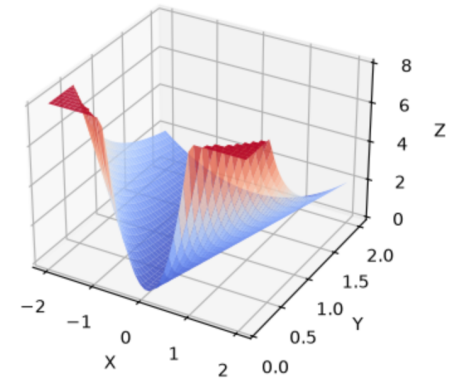

```
[[-2.  0.]
 [ 0. -2.]]
```

c) f(x,y) = xy




```
[[0. 1.]
 [1. 0.]]
```

d) f(x,y) = x² / y




```
[[2. 0.]
 [0. 0.]]
```

```
a) is Convex because all diagonal curvatures are positive.
b) is Concave because all diagonal curvatures are negative.
c) is Neither because there is a saddle point.
d) is Convex because the matrix is positive where y > 0.
```

P7 – Python Coding [20 points]

Find the minima of $f(x) = x^2 + k\cos^2(x)$ with $k > 0$. How many minima? Will the number of minima depend on k?

Python Programming: Plot the function for -30 < x < 30 using for k=1,2,3. Show the minima for each case on the plot.

Your Code

```
# (minima listed below). There are 1 to multiple minima based on the value of k.
# This is because x^2 wants to create a bowl shaped parabola and
# cos^2(x) wants to create a wave. The larger the value of k, the more
# amplified the wave across the curve which creates more local minima.

def f(x, k):return x**2 + k * np.cos(x)**2

x_vals = np.linspace(-30, 30, 6001)
ks = [1,2,3, 10]

plt.figure(figsize=(12, 6))

for k in ks:
    y_vals = f(x_vals, k)

    plt.plot(x_vals, y_vals, label=f'k = {k}', lw=2)
    minima_indices = np.where((y_vals[1:-1] < y_vals[:-2]) & (y_vals[1:-1] < y_vals[2:]))[0] + 1
    minima_x = x_vals[minima_indices]
    minima_y = y_vals[minima_indices]
    plt.scatter(minima_x, minima_y, color='red', zorder=5, s=50)
```

```
    print(f"Minima for k = {k}")
    print(f"Total minima found: {len(minima_x)}")
    for idx in range(len(minima_x)):
        print(f"  Minimum {idx+1}: x = {minima_x[idx]:.4f}, f(x) = {minima_y[idx]:.4f}")
    print()

plt.scatter([], [], color='red', s=50, label='Local Minima')
plt.title(r"Plot of $f(x) = x^2 + k\cos^2(x)$")
plt.xlabel("x")
plt.ylabel("f(x)")
plt.xlim(-10, 10)
plt.ylim(-1, 40)
plt.grid(True, linestyle="--", alpha=0.5)
plt.legend()

print(f'')
plt.show()
```

Run the code and insert the result in the following box.

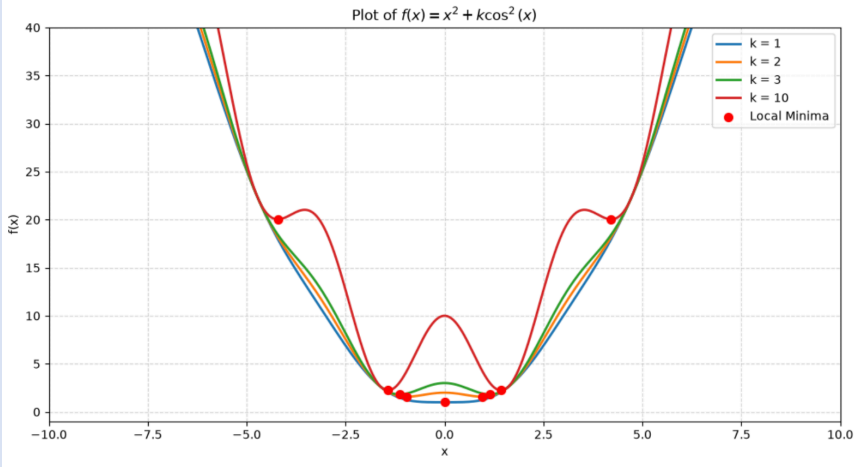
Plots

```
Minima for k = 1
Total minima found: 1
  Minimum 1: x = 0.0000, f(x) = 1.0000

Minima for k = 2
Total minima found: 2
  Minimum 1: x = -0.9500, f(x) = 1.5792
  Minimum 2: x = 0.9500, f(x) = 1.5792

Minima for k = 3
Total minima found: 2
  Minimum 1: x = -1.1400, f(x) = 1.8228
  Minimum 2: x = 1.1400, f(x) = 1.8228

Minima for k = 10
Total minima found: 4
  Minimum 1: x = -4.2100, f(x) = 20.0427
  Minimum 2: x = -1.4300, f(x) = 2.2418
  Minimum 3: x = 1.4300, f(x) = 2.2418
  Minimum 4: x = 4.2100, f(x) = 20.0427
```



The End